



**POLITECNICO DI BARI**

**DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELL'INFORMAZIONE  
MASTER DEGREE IN AUTOMATION ENGINEERING**

---

**Mobile robotics**

Prof. Luca De Cicco

**Title:**

**Autonomous Navigation System for Vision-Based Human Target Following  
in ROS 2 using ultralytics YOLO model**

**Student:**

Domenico D'Introno

ACADEMIC YEAR 2024-2025



## Contents

|          |                                      |           |
|----------|--------------------------------------|-----------|
| <b>1</b> | <b>Robot</b>                         | <b>2</b>  |
| <b>2</b> | <b>Environment</b>                   | <b>3</b>  |
| <b>3</b> | <b>YOLO Computer Vision Model</b>    | <b>4</b>  |
| <b>4</b> | <b>Interfaces</b>                    | <b>5</b>  |
| <b>5</b> | <b>Custom Nodes</b>                  | <b>6</b>  |
| 5.1      | Perceptor . . . . .                  | 6         |
| 5.2      | Localizer . . . . .                  | 8         |
| 5.3      | Navigator . . . . .                  | 10        |
| 5.4      | Initial position publisher . . . . . | 10        |
| <b>6</b> | <b>Launch File</b>                   | <b>11</b> |



# 1 Robot



## 2 Environment

### 3 YOLO Computer Vision Model



## 4 Interfaces

## 5 Custom Nodes

This section describes the design and implementation of the three custom ROS 2 nodes: the **Perceptor**, the **Localizer**, and the **Navigator**. Together, they form a pipeline allowing the robot to detect a human in its camera feed, to estimate their 3D position relative to the robot, and to autonomously navigate towards that position using the Nav2 framework.

### 5.1 Perceptor

The *Perceptor* node is responsible for detecting humans in the camera image stream. It leverages the YOLOv8 neural network model to perform real-time object detection.

#### Logic pipeline

1. **Subscribes** to the raw RGB image topic `/camera/image_raw`.
2. Applies YOLO to detect the human target.
3. Computes the centroid of the detected bounding box.
4. **Publishes** the centroid as a custom message `CentroidCoords` on `/centroid_coords`.

#### Code analysis

```
1 import rclpy
2 from rclpy.node import Node
3
4 from sensor_msgs.msg import Image
5 from doom_interfaces.msg import CentroidCoords
6
7 from cv_bridge import CvBridge
8
9 from ultralytics import YOLO
10
11 class Detector(Node):
12
13     def __init__(self):
14         super().__init__('perceptor')
15
16         self.model = YOLO("yolov8m.pt")
17
18         self.bridge = CvBridge()
19
20         self.subscriber_ = self.create_subscription(
21             Image,
22             '/camera/image_raw',
23             self.detect,
24             10
25         )
26
27         self.publisher_ = self.create_publisher(
28             CentroidCoords,
29             '/centroid_coords',
30             10
31         )
32
```

```
33     self.get_logger().info(f"Detection node ready \n")
34
35     def detect(self, msg):
36
37         # Convert a sensor_msgs\Image message in a format that can be used by
38         # ↳ the YOLO model
39         cv_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding='
40         ↳ passthrough')
41
42         # Detect a person (class : 0) inside the camera image, use gpu
43         results = self.model(cv_image, device="cuda", classes=[0], verbose=
44         ↳ False)
45
46         # Check that a person has actually been detected, otherwise publish
47         # ↳ the 'error centroid'
48         if len(results[0].boxes) > 0:
49             [x_min, y_min, x_max, y_max] = results[0].boxes.xyxy[0]
50
51             # Compute centroid (top 25% to avoid targeting the void space
52             # ↳ between the legs)
53             x_centroid = int((x_min + x_max) / 2)
54             y_centroid = int(y_min + (y_max-y_min) * 0.25)
55
56             centroid = CentroidCoords()
57             centroid.u = x_centroid
58             centroid.v = y_centroid
59             centroid.header = msg.header
60         else:
61             centroid = CentroidCoords()
62             centroid.u = -1
63             centroid.v = -1
64             centroid.header = msg.header
65
66         self.publisher_.publish(centroid)
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

The class **constructor** initializes the **perceptor** node: it loads the YOLOv8 model, initializes the CV bridge, which converts between ROS2 Image messages and OpenCV images, subscribes to camera topic, and sets up a publisher for the detected centroid coordinates. The **detect** callback function



runs person detection on the received camera image using YOLO, extracts the bounding box centroid, and publishes it as `CentroidCoords`.

Note that the `model` function takes as arguments `device="cuda"` to specify that the GPU will be used to run the model, plus, the `classes=[0]` argument narrows the model research to just persons.

An important design choice is to compute the person pixel centroid as the **top 25% of its silhouette**, fixing a bug where inconsistent depth measurements and mislocalization occurred when the centroid fell between the legs.

Also, if no person is detected, the node publishes an **'error centroid'**, meaning a centroid with both pixels coordinates set to -1. This strategy allows the downstream nodes to identify target loss. Finally, the script includes the standard Python entry point structure to make it executable as a ROS 2 node.

The `main` function handles the node's lifecycle. It begins by initializing the ROS 2 client library using `rclpy.init()`. Then, it instantiates the `Detector` node object. The core of the function is the `try/except/finally` block: it enters the ROS 2 event loop, keeping the node alive to receive messages and execute callbacks until the node is shut down by the user, then it ensures graceful shutdown: upon interruption, the `node.destroy_node()` method is called to release ROS 2 resources, followed by `rclpy.shutdown()` to close the client library communication.

The `if __name__ == '__main__':` check is a standard Python idiom. It ensures that the `main()` function is called only when the script is executed directly.

## 5.2 Localizer

The *Localizer* node acts as the bridge between 2D image perception and the 3D world. Its primary role is to estimate the 3D position of the detected human target in the camera's coordinate frame by combining the pixel coordinates from the *Perceptor* with depth information from the robot's depth camera.

### Logic pipeline

1. **Subscribes** to the centroid coordinates topic `/centroid_coords` published by *Perceptor*.
2. **Subscribes** to the depth point cloud topic `/camera/depth/points`.
3. Uses an **Approximate Time Synchronizer** to receive pairs of centroid and point cloud messages that are close in time.
4. In the callback, checks if the received centroid is valid or an 'error centroid'.
5. If the centroid is valid, it extracts the corresponding 3D point (x, y, z) from the point cloud using the pixel coordinates (u, v) to compute the corresponding index.
6. If the centroid is an 'error centroid', it generates a predefined 'error point'.
7. **Publishes** the resulting 3D point (either valid or error) as a `geometry_msgs/PointStamped` message on the `/target_position_camera` topic.

### Code analysis

```
1 import rclpy
2 from rclpy.node import Node
3
4 from sensor_msgs.msg import PointCloud2
5 from geometry_msgs.msg import PointStamped
6 from doom_interfaces.msg import CentroidCoords
7
```

```
8 from message_filters import Subscriber, ApproximateTimeSynchronizer
9 import sensor_msgs_py.point_cloud2 as pc2
10
11 import numpy as np
12
13
14 class Localizer(Node):
15
16     def __init__(self):
17         super().__init__('localizer')
18
19         self.sub_centroid = Subscriber(self, CentroidCoords, '/centroid_coords
20         ↪ ')
21         self.sub_depth = Subscriber(self, PointCloud2, '/camera/depth/points')
22
23         # Initialize approximate time synchronizer to synchronize the two
24         ↪ different messages
25         ats = ApproximateTimeSynchronizer(
26             [self.sub_centroid, self.sub_depth],
27             queue_size=10,
28             slop=0.1
29         )
30         ats.registerCallback(self.localize)
31
32         self.publisher_ = self.create_publisher(
33             PointStamped,
34             '/target_position_camera',
35             10
36         )
37
38         self.get_logger().info(f"Localization node ready \n")
39
40     def localize(self, msg_centroid, msg_depth):
41
42         u = msg_centroid.u
43         v = msg_centroid.v
44
45         # If the 'error centroid' has been received, forward the 'error point
46         ↪ ', otherwise
47         # normally extract the depth point from the pointcloud
48         if u < 0 or v < 0:
49             x = 0.0
50             y = 0.0
51             z = -1.0
52         else:
53             height = msg_depth.height
54             width = msg_depth.width
55             # The pointcloud has linear indexing
56             index = v * width + u
57             points_array = pc2.read_points_numpy(msg_depth, field_names=("x",
58             ↪ "y", "z"))
59             x, y, z = points_array[index]
60
61         msg = PointStamped()
62         msg.point.x = float(x)
63         msg.point.y = float(y)
```

```
60     msg.point.z = float(z)
61     msg.header = msg_depth.header
62
63     self.publisher_.publish(msg)
64
65
66
67
68 def main(args=None):
69     rclpy.init(args=args)
70     node = Localizer()
71     try:
72         rclpy.spin(node)
73     except KeyboardInterrupt:
74         pass
75     finally:
76         node.destroy_node()
77         rclpy.shutdown()
78
79
80 if __name__ == '__main__':
81     main()
```

The class **constructor** initializes the **localizer** node. It sets up two `message_filters.Subscriber` instances, one for the `CentroidCoords` messages from the *Perceptor* and another for the `PointCloud2` messages from the depth sensor. Crucially, it initializes an `ApproximateTimeSynchronizer`. This synchronizer ensures that the `localize` callback function is only called when a pair of centroid and point cloud messages with closely matching timestamps (within the `slop` parameter value, here 0.1 seconds) are received. This synchronization is vital for accurately associating a 2D detection with its corresponding 3D depth data. Finally, the constructor creates a publisher for the resulting `PointStamped` message.

The `localize` callback is the core logic. It receives the synchronized centroid and point cloud messages. It first checks if the centroid coordinates `u` and `v` are negative, which signifies the 'error centroid' sent by the *Perceptor* when no person is detected: in this case, it constructs and publishes a `PointStamped` message with `z = -1.0` to propagate this error signal downstream. If the centroid is valid, it calculates the linear `index` corresponding to the pixel coordinates within the point cloud structure (using `v * width + u`). It then uses the `pc2.read_points_numpy` function to read the *entire* point cloud into a NumPy array and extracts the single 3D point (`x`, `y`, `z`) at the calculated `index`. Finally, it constructs a `PointStamped` message using the determined coordinates (either valid 3D point or error point), sets its header using the timestamp from the input `msg.depth` for consistency, and publishes it.

Finally, the script includes the standard Python entry point structure.

### 5.3 Navigator

### 5.4 Initial position publisher

## 6 Launch File